# SIMATS ENGINEERING

## ASSESSMENT TOOL 3: Mock Interview

**COURSE NAME: COMPUTER VISION** **COURSE CODE: ITA05**

**COURSE OUTCOME COVERED**

**CO5:** *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.* (BTL 6)

Assessment Tool Weightage: 10%

**Mock Interview Questions**

**A.R.Yuvansri**

**192421029**

# 1. Live Video Feed Text & Shape Overlay Design

## Full Question Prompt:

*You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.*

## Solution & Technical Implementation:

To overlay shapes and text on a real-time frame without introducing latency, operations must be performed directly in-place on the memory buffer of each frame prior to display or encoding.

**Implementation Steps & Key OpenCV Functions:**

- **Frame Capture:** Use cv2.VideoCapture() to fetch real-time frames from a camera or stream.
- **Region Computation:** Calculate target pixel coordinates (bounding box points, label origins) based on detection logic or user inputs.
- **Shape Drawing (cv2.rectangle):** Draws bounding boxes around objects (cv2.rectangle(img, pt1, pt2, color, thickness)).
- **Point Highlights (cv2.circle):** Highlights point features, centroids, or keypoints (cv2.circle(img, center, radius, color, thickness)).
- **Boundary Drawing (cv2.polylines):** Draws multi-point boundaries or trajectories (cv2.polylines(img, [pts], isClosed, color, thickness)).
- **Text Rendering (cv2.putText):** Renders text labels such as Class Name, ID, Confidence score (cv2.putText(img, text, org, fontFace, fontScale, color, thickness, lineType)). Set lineType=cv2.LINE_AA for anti-aliased text.

**Efficiency Considerations:**

- **In-Place Modification:** Pass frames directly without making unnecessary array copies (img.copy()), saving memory allocation overhead.
- **Resolution Independence:** Compute overlay positions scaled to frame width and height so text and shapes remain proportional across varying resolutions.

# 2. Feature Detection and Visualization Approach

## Full Question Prompt:

*You are asked to highlight key features in an image. Design a feature detection and visualization approach, explaining the techniques and their suitability.*

## Solution & Technical Implementation:

Highlighting key features requires extracting distinctive image points (edges, corners, or scale-invariant visual markers) that resist changes in scale, illumination, and rotation.

**Technique Selection & Suitability:**

- **Corner & Point Features (FAST / ORB):** Suitable for real-time performance, low computational cost, and rotation invariance. Functions: cv2.ORB_create() paired with orb.detectAndCompute(). Visualization: cv2.drawKeypoints() draws circles and vector directions representing key feature scale and orientation.
- **Scale and Rotation Invariant Features (SIFT):** Suitable for high-accuracy tasks involving extreme scale shifts or occlusion. Functions: cv2.SIFT_create() paired with sift.detectAndCompute().
- **Edge Features (Canny Edge Detection):** Suitable for structural outline identification. Functions: cv2.Canny(image, threshold1, threshold2). Visualization: cv2.bitwise_and() overlays detected edge maps in distinct colors onto the original image canvas.

# 3. Visual Image Difference Comparison

## Full Question Prompt:

*You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.*

## Solution & Technical Implementation:

Simply subtracting pixels (ImageA - ImageB) fails under real-world conditions due to minor alignment shifts, environmental lighting variations, and sensor noise. A robust visual comparison pipeline isolates structural changes from noise.

**Implementation Steps:**

- **Alignment & Grayscale Conversion:** Align images to identical dimensions and convert them to grayscale via cv2.cvtColor().
- **Denoising:** Apply a Bilateral Filter (cv2.bilateralFilter) to smooth out high-frequency sensor noise while preserving sharp structural edges.
- **Structural & Difference Computation:** Compute Structural Similarity Index (SSIM) to evaluate structural changes, and Absolute Pixel Difference (cv2.absdiff) to capture magnitude changes.
- **Binarization & Cleaning:** Apply Otsu's thresholding (cv2.threshold) followed by morphological closing (cv2.morphologyEx) to merge fragmented noise into coherent change masks.
- **Visualization Overlay:** Extract change contours (cv2.findContours) and draw bounding boxes (cv2.rectangle) or a colored mask layer (cv2.addWeighted) over modified regions.

**Justification of Approach:**

Combining SSIM with morphological operations filters out ambient lighting fluctuations, restricting visual highlights strictly to real, structural object variations.

# 4. Analysis & Remediation of Face Detection False Positives

## Full Question Prompt:

*A face detection system produces multiple false positives. Analyze the issue and propose improvements, explaining how each change enhances accuracy.*

## Solution & Technical Implementation:

Root Cause Analysis: False positives in face detection occur when non-face background clutter (textures, shadows, wall patterns) mimics facial characteristics under static detection thresholds or fixed-scale sliding windows.

**Proposed Improvements & Enhancements:**

- **Increase Confidence & Intersection Thresholds:** Raise detection confidence scores and Non-Maximum Suppression (NMS) IoU thresholds. Enhancement: Eliminates weak, highly overlapping candidate boxes that capture background noise.
- **Multi-Scale Image Pyramid Tuning:** Adjust scale factors (e.g., from 1.05 to 1.2 in Haar cascades) and set realistic minimum/maximum face size constraints (minSize, maxSize). Enhancement: Prevents the detector from searching scales that are physically improbable within the camera setup.
- **Preprocessing & Contrast Normalization:** Apply CLAHE (Contrast Limited Adaptive Histogram Equalization) before detection. Enhancement: Balances harsh directional lighting and extreme shadows, reducing false edge triggers.
- **Modern Model Migration:** Upgrade legacy Viola-Jones/Haar Cascades to Deep Learning-based detectors (such as RetinaFace or MediaPipe Face Detector). Enhancement: Convolutional features model complex spatial geometry better than manual Haar features, reducing false positive rates.

# 5. Adapting Facial Expression Recognition to Head Movements

## Full Question Prompt:

*A facial expression system fails with slight head movements. Analyze the limitation and propose modifications, justifying your approach.*

## Solution & Technical Implementation:

Root Cause Analysis: Facial expression recognition models often rely on absolute 2D landmark coordinates (e.g., distance between lips or eyes). When a user tilts or turns their head (yaw, pitch, roll), 2D distance values distort even if the expression remains unchanged.

**Proposed Modifications & Justification:**

- **Pose Normalization via Affine Alignment:** Estimate 2D/3D facial landmarks (MediaPipe Face Mesh), compute an alignment matrix based on eye/nose anchors, and apply an affine transform (cv2.getAffineTransform / cv2.warpAffine) to warp the face into a canonical frontal view before classification. Justification: Removes head pose variations (yaw/roll), presenting the model with a standardized frontal face.
- **Relative Geometry & Normalized Ratios:** Instead of absolute pixel distances, calculate normalized Euclidean distances relative to fixed facial baselines (e.g., lip stretch distance divided by inter-ocular distance). Justification: Keeps feature metrics scale- and pose-invariant across dynamic movement.
- **3D Facial Mesh Integration:** Replace 2D landmark tracking with 3D landmark tracking. Justification: 3D coordinates preserve spatial depth during head rotation, preventing perspective compression from skewing facial muscle measurements.